INSTITUTO FED. DE EDUCAÇÃO, CIÊNC. E TEC. DE PERNAMBUCO
CURSO: TEC. EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: ALGORITMOS E ESTRUTURAS DE DADOS
PROFESSOR: RAMIDE DANTAS
ASSUNTO: REVISÃO – ARRAYS, STRINGS E RECURSÃO

Prática 01

Qualquer IDE pode ser usada **desde seja possível fazer depuração (debug).** Sugestões:

- Eclipse com CDT + MinGW (local; instalação meio complicada).
- CLion (local; meio pesado, requer licença de estudante).
- Replit.com (online; requer acesso constante a internet).
- OnlineGDB (online; meio limitado mas usável).

Parte 1: Arrays e Recursão

Problema 1: Dado um array (vector) de inteiros e um valor N (N >= 0), escreva uma função que desloque os elementos do array N posições à direita. Os elementos cuja nova posição extrapolem o tamanho do array são colocados ao início, por exemplo, o array:

3	5	8	10	1	7	9	4

Com valor N (descolamento) = 3, o array ao final ficaria:

7	9	4	3	5	8	10	1

Uma implementação boa deve usar apenas um *loop* (*for*, *while*) e resto da divisão (operador %) para calcular as novas posições dos elementos. Pode ser usado um array/vector auxiliar. Use o código abaixo como base; chame o arquivo **rotate.cpp**. Pense também como deixar o código genérico o suficiente para lidar com valores negativos de N.

```
#include <iostream>
#include <vector>

using namespace std;

void rotate(vector<int> &arr, int n) {
    // TODO
}

int main() {
    vector<int> arr1 = {1, 2, 3, 4, 5, 6, 7, 8};
    vector<int> arr2 = {8, 7, 6, 5, 4, 3, 2, 1};
    vector<int> arr3 = {3, 5, 8, 10, 1, 7, 9, 4};

    rotate(arr1, 2);
    for (auto & v : arr1) cout << v << " "; // 7 8 1 2 3 4 5 6
    cout << endl;

    rotate(arr2, 5);
    for (auto & v : arr2) cout << v << " "; // 5 4 3 2 1 8 7 6
    cout << endl;

    rotate(arr3, 3);
    for (auto & v : arr3) cout << v << " "; // 7 9 4 3 5 8 10 1
    cout << endl;

    return 0;
}
```

Desafio (Opcional): Adapte e submeta sua solução ao problema 189 no LeetCode.

Problema 2: Dado um array (ou vector) de números inteiros, escreva uma função **recursiva** que retorna o **maior** número do array.

Array de entrada:

10	1	5	3	12	7	4	6

Resultado (saída): 12

Use o código abaixo como base, chame o arquivo **maior.cpp**.

```
#include <iostream>
#include <vector>

using namespace std;

int maior(const vector<int> &arr, int n) {
    // TODO
}

int main() {
    vector<int> arr1 = {1, 1, 1, 1, 1, 1, 1, 2};
    vector<int> arr2 = {-1, -2, -3, -4, -5, -6};
    vector<int> arr3 = {10, 1, 5, 3, 12, 7, 4, 6};

    cout << maior(arr1, arr1.size()) << endl; // 2
    cout << maior(arr2, arr2.size()) << endl; // -1
    cout << maior(arr3, arr3.size()) << endl; // 12
    return 0;
}
```

Parte 2: Strings, Arrays e Recursão

Problema 1: Dada uma string com caracteres minúsculos ('a' a 'z'), reescrever essa string de forma que as letras existentes sejam "reordenadas", isto é, apareçam em ordem alfabética. Se houver repetições no string original, devem ocorrer no string final. Exemplos:

- "casa" → "aacs"
- "escola" → "acelos"
- "ifpe" → "efip"
- "banana" → "aaabnn"

Use o código abaixo; chame o arquivo **strings.cpp**. **Adicione seu nome como casos de teste.**

ATENÇÃO: resolver apenas com arrays/`vector`; **não usar métodos como `std::sort()`**, etc.

```
#include <iostream>
#include <vector>

using namespace std;

string reordenar(const string &str) {
    // TODO
}

int main() {
    cout << reordenar(string("casa")) << endl;
    cout << reordenar(string("escola")) << endl;
    cout << reordenar(string("ifpe")) << endl;
    cout << reordenar(string("banana")) << endl;
}
```

Desafio (Opcional): Adapte sua solução para resolver o problema 1276 no Beecrowd.

Problema 2: Implemente uma função recursiva que receba uma string e retorne uma nova string com os caracteres invertidos (isto é, na ordem contrária à original). Exemplos:

- "recursao" → "oasrucer"
- "banana" → "ananab"

Use o código abaixo; chame de **reverse.cpp**. **Adicione seu nome como caso de teste.**

ATENÇÃO: Não use usar *loops* (for, while) nem funções como reverse() ou std::reverse().

```
#include <iostream>
#include <vector>
using namespace std;

string inverter(const string &s) {
    // TODO
}

int main() {
    cout << inverter("recursao") << endl;  // oasrucer
    cout << inverter("banana") << endl;    // ananab
    cout << inverter("ifpe") << endl;      // epfi
}
```

Desafio (opcional): diga se a uma string é um palíndromo usando apenas com recursão.